💻 DAY 1: MASTER STEP-BY-STEP COMMANDS & SETUP RUNBOOK

Complete Practical Execution Guide (Linux Setup, Shell Automation, SSH & Intel oneAPI/HPC Toolkit)

Course Code: `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Target Session:** Day 1 (Full 6 Hours: 3h Morning Concept + 3h Afternoon Hands-On Lab)
Primary Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)

📌 Executive Architecture & Environment Strategy

This runbook contains **every single command** required to set up, configure, and execute the entire Day 1 hands-on curriculum from scratch on: 1. **Ubuntu Linux / WSL2** (Trainer Laptop & Lab Workstations) 2. **Google Colab** (Zero-friction browser sandbox for 60 students) 3. **Intel Developer Cloud (`idc-vm`)** (Enterprise 4th Gen Xeon Sapphire Rapids)

```
+-------------------------------------------------------------------------------------------------+
|                                  THE 8-STAGE COMMAND PIPELINE                                   |
+-------------------------------------------------------------------------------------------------+
| STAGE 0: Linux System Package Setup & Preflight Verification                                    |
| STAGE 1: Hardware Introspection & Silicon Architecture Probing (Cores, Caches, AVX-512, AMX)    |
| STAGE 2: Linux Directory Tree Hierarchy & File Management (pwd, mkdir, heredoc, cat, grep)      |
| STAGE 3: Permissions, Security & Environment Variables ($PATH, export, chmod 755/644/600)      |
| STAGE 4: Automated Shell Scripting (Writing & Executing hpc_node_audit.sh)                      |
| STAGE 5: Secured Shell (SSH) Key Generation & Remote Access (ed25519, authorized_keys, scp)     |
| STAGE 6: Intel oneAPI Toolkit Installation, Environment Setup & C++/SYCL Vector Addition        |
| STAGE 7: Intel HPC Toolkit Setup & Intel MPI Multi-Process Supercomputing Simulation            |
+-------------------------------------------------------------------------------------------------+
```

🚀 STAGE 0: Linux System Package Setup & Preflight Verification

Run these commands on a fresh Ubuntu/WSL machine to install core development tools:

```
# 1. Verify operating system distribution and kernel release
cat /etc/os-release
uname -sr

# 2. Identify the active user and machine hostname
whoami
hostname

# 3. Update local package repository lists
sudo apt-get update -y
```

```
# 4. Install essential Linux development utilities, compilers, and networking tools
sudo apt-get install -y build-essential g++ gdb wget curl gnupg openssh-client

# 5. Verify presence of core build utilities
gcc --version
g++ --version
ssh -V
```

🔬 STAGE 1: Hardware Introspection & Silicon Architecture

Maps to Syllabus Topics 1 & 2: Introduction to HPC & Intel HPC Architecture

```
# 1. Print comprehensive CPU architecture summary
lscpu

# 2. Extract CPU Model, Socket count, and Hardware Threads per Core
lscpu | grep -E "Model name|Socket\(s\)|Core\(s\) per socket|Thread\(s\) per core|NUMA"

# 3. Probe for hardware vector and matrix acceleration flags
# Look for avx512f (512-bit SIMD) and amx_tile/amx_bf16 (Intel AMX Matrix Engines)
lscpu | grep -E "avx512|amx"

# 4. Inspect physical memory (RAM) and Swap allocation
free -h

# 5. Inspect cache hierarchy (L1 data, L1 instruction, L2, and shared L3 Last Level Cache)
lscpu | grep -E "L1d|L1i|L2|L3"

# 6. Check root filesystem storage capacity
df -h /
```

📂 STAGE 2: Linux Directory Tree Hierarchy & File Management

Maps to Syllabus Topic 3: Linux Basics (File system, navigation, commands)

```
# 1. Print the Current Working Directory
pwd

# 2. Create the standardized high-performance project hierarchy
mkdir -p ~/hpc_day01/{scripts,data,logs,bin}

# 3. Navigate into the scripts directory and verify location
cd ~/hpc_day01/scripts
pwd

# 4. Generate cluster configuration file using Heredoc redirection
cat << 'EOF' > cluster_config.env
# ================================================================================
# Intel HPC Cluster Environment Configuration
# ================================================================================
CLUSTER_NAME="GLA_Intel_HPC_Cluster"
```

```
HEAD_NODE_IP="10.100.1.1"
COMPUTE_NODES=("node01" "node02" "node03" "node04")
MAX_JOBS=16
DEFAULT_PARTITION="standard"
SCRATCH_STORAGE="/scratch/gla_user"
EOF

# 5. Inspect file creation and metadata
ls -lh cluster_config.env

# 6. Print file contents to terminal
cat cluster_config.env

# 7. Search text patterns inside the file using grep
grep "COMPUTE_NODES" cluster_config.env
grep "SCRATCH" cluster_config.env

# 8. Copy file to create a backup
cp cluster_config.env cluster_config.env.bak
ls -l cluster_config.env*

# 9. Move the backup file into the data directory
mv cluster_config.env.bak ~/hpc_day01/data/production.env
ls -l ~/hpc_day01/data/

# 10. Recursively search filesystem for all .env configuration files
find ~/hpc_day01 -name "*.env"
```

STAGE 3: Permissions, Security & Environment Variables

Maps to Syllabus Topic 3: Linux Basics (Permissions, environment variables, PATH)

```
# 1. Print active shell environment variables
echo "User: $USER | Shell: $SHELL | Hostname: $(hostname)"

# 2. View current executable search path ($PATH)
echo $PATH

# 3. Create a custom environment variable pointing to the HPC project root
export GLA_HPC_ROOT="$HOME/hpc_day01"
echo "Root workspace set to: $GLA_HPC_ROOT"

# 4. Prepend custom bin directory to $PATH
export PATH="$GLA_HPC_ROOT/bin:$PATH"
echo "Updated PATH: $PATH"

# 5. Inspect directory permissions in octal format
ls -ld ~/hpc_day01/scripts

# 6. Set secure production permissions:
# Owner: read/write/execute (7), Group & Others: read/execute (5)
chmod 755 ~/hpc_day01/scripts
```

STAGE 4: Automated Shell Scripting for HPC (Step-by-Step Progressive Drill)

Maps to Syllabus Topic 4: Shell Scripting for Automation

Level 1: Shebang, Variables & Command Substitution (`01_basic_syntax_and_vars.sh`)

```
cat << 'EOF' > ~/hpc_day01/scripts/01_basic_syntax_and_vars.sh
#!/usr/bin/env bash
# RULE: NO SPACES around '=' when assigning variables!
STUDENT_NAME="GLA_Supercomputing_Student"
WORKSHOP_DAY=1
CURRENT_TIME=$(date '+%Y-%m-%d %H:%M:%S')

echo "============================================================"
echo "  STEP 1: BASH BASICS & ENVIRONMENT VARIABLES"
echo "============================================================"
echo "Welcome, ${STUDENT_NAME}!"
echo "Workshop Day     : ${WORKSHOP_DAY}"
echo "Current Hostname : $(hostname)"
echo "Current Time     : ${CURRENT_TIME}"
echo "============================================================"
EOF

chmod +x ~/hpc_day01/scripts/01_basic_syntax_and_vars.sh
~/hpc_day01/scripts/01_basic_syntax_and_vars.sh
```

Level 2: Conditionals (if/else), Directory Checks & Exit Codes (`02_conditions_and_exit_codes.sh`)

```
cat << 'EOF' > ~/hpc_day01/scripts/02_conditions_and_exit_codes.sh
#!/usr/bin/env bash
WORKSPACE_DIR="${HOME}/hpc_day01/data"

echo "============================================================"
echo "  STEP 2: CONDITIONALS & EXIT STATUS CODES"
echo "============================================================"

if [ -d "$WORKSPACE_DIR" ]; then
    echo "[+] Found existing directory: $WORKSPACE_DIR"
else
    echo "[-] Directory not found. Creating it now..."
    mkdir -p "$WORKSPACE_DIR"
    echo "[+] mkdir command completed with exit code: $?"
fi

SAMPLE_FILE="${WORKSPACE_DIR}/sample.txt"
echo "HPC Batch Processing initialized on $(date)" > "$SAMPLE_FILE"

if [ -f "$SAMPLE_FILE" ]; then
    echo "[+] Verified file creation: $SAMPLE_FILE"
    echo "[+] File size: $(wc -c < "$SAMPLE_FILE") bytes"
fi
echo "============================================================"
```

```
EOF

chmod +x ~/hpc_day01/scripts/02_conditions_and_exit_codes.sh
~/hpc_day01/scripts/02_conditions_and_exit_codes.sh
```

Level 3: Loops, Node Arrays & Output Redirection (`03_batch_loop_automation.sh`)

```
cat << 'EOF' > ~/hpc_day01/scripts/03_batch_loop_automation.sh
#!/usr/bin/env bash
COMPUTE_NODES=("node-cpu-01" "node-cpu-02" "node-gpu-01" "node-ai-gaudi")
LOG_FILE="${HOME}/hpc_day01/logs/cluster_dispatch.log"

> "$LOG_FILE"

echo "============================================================"
echo "  STEP 3: CLUSTER BATCH LOOP & AUTOMATION"
echo "============================================================"
echo "[*] Dispatching tasks across ${#COMPUTE_NODES[@]} compute nodes..."

for node in "${COMPUTE_NODES[@]}"; do
    TIMESTAMP=$(date '+%H:%M:%S')
    echo "[${TIMESTAMP}] Dispatched batch job to ${node} -> Status: ACTIVE" >> "$LOG_FILE"
    echo "  • Node ${node}: Assigned Batch Worker Task"
done

echo "------------------------------------------------------------"
echo "[+] All tasks dispatched! Persisted to: $LOG_FILE"
cat "$LOG_FILE"
echo "============================================================"
EOF

chmod +x ~/hpc_day01/scripts/03_batch_loop_automation.sh
~/hpc_day01/scripts/03_batch_loop_automation.sh
```

Level 4: Full Node Hardware & Memory Audit Capstone (`hpc_node_audit.sh`)

```
cat << 'EOF' > ~/hpc_day01/scripts/hpc_node_audit.sh
#!/usr/bin/env bash
# ==============================================================================
# Script Name : hpc_node_audit.sh
# Purpose     : Automated Compute Node Hardware & Software Health Audit
# Author      : GLA University HPC Masterclass (Lead: Shivam Gupta)
# ==============================================================================

set -euo pipefail

LOG_DIR="${HOME}/hpc_day01/logs"
mkdir -p "$LOG_DIR"
TIMESTAMP="$(date '+%Y%m%d_%H%M%S')"
LOG_FILE="${LOG_DIR}/node_audit_${TIMESTAMP}.log"

{
    echo "============================================================"
    echo "       INTEL HPC NODE HARDWARE & SOFTWARE AUDIT             "
    echo "============================================================"
```

```
    echo "Timestamp          : $(date '+%Y-%m-%d %H:%M:%S')"
    echo "Host Node Name     : $(hostname)"
    echo "Active User        : $(whoami)"
    echo "Linux OS Kernel    : $(uname -r)"
    echo "Architecture       : $(uname -m)"
    echo "CPU Model          : $(lscpu | grep 'Model name:' | head -n1 | sed 's/Model name:[ \t]*
    echo "Available Cores    : $(nproc)"
    echo "Total Memory (RAM) : $(free -h | awk '/^Mem:/ {print $2}')"
    echo "Available Memory   : $(free -h | awk '/^Mem:/ {print $7}')"
    echo "Filesystem / Free  : $(df -h / | awk 'NR==2 {print $4}')"
    echo "------------------------------------------------------------"
    echo "Hardware Acceleration Probing:"

    if grep -q "avx512" /proc/cpuinfo; then
        printf "  %-20s: [SUPPORTED]\n" "AVX-512 Vector"
    else
        printf "  %-20s: [NOT DETECTED]\n" "AVX-512 Vector"
    fi

    if grep -q "amx" /proc/cpuinfo; then
        printf "  %-20s: [SUPPORTED]\n" "Intel AMX Matrix"
    else
        printf "  %-20s: [NOT DETECTED]\n" "Intel AMX Matrix"
    fi

    echo "------------------------------------------------------------"
    echo "Intel oneAPI & Development Tools In PATH:"
    for tool in bash ssh icpx ifx mpirun sycl-ls g++; do
        if command -v "$tool" >/dev/null 2>&1; then
            printf "  %-12s: AVAILABLE (%s)\n" "$tool" "$(command -v "$tool")"
        else
            printf "  %-12s: NOT FOUND IN PATH\n" "$tool"
        fi
    done
    echo "============================================================"
} | tee "$LOG_FILE"

echo ""
echo "[INFO] Audit successfully completed and persisted to: $LOG_FILE"
EOF

# Grant executable permission to the script
chmod u+x ~/hpc_day01/scripts/hpc_node_audit.sh

# Execute the automated script
~/hpc_day01/scripts/hpc_node_audit.sh

# Verify log file
ls -la ~/hpc_day01/logs/
```

🔒 STAGE 5: Secured Shell (SSH) Key Generation & Remote Access

Maps to Syllabus Topic 5: Secured Shell (SSH)

```
# 1. Verify SSH client presence
ssh -V

# 2. Create secure .ssh directory with strict permissions
mkdir -p ~/.ssh
chmod 700 ~/.ssh

# 3. Generate an industrial-grade Ed25519 asymmetric cryptographic key pair
ssh-keygen -t ed25519 -C "student@gla_hpc_cluster" -f ~/.ssh/id_ed25519_gla -N ""

# 4. Inspect generated keys and verify strict file permissions
ls -la ~/.ssh/id_ed25519_gla*

# 5. Ensure private key permissions are strictly 600 (read/write only by owner):
chmod 600 ~/.ssh/id_ed25519_gla
chmod 644 ~/.ssh/id_ed25519_gla.pub

# 6. View Public Key string (Safe to distribute to server authorized_keys)
cat ~/.ssh/id_ed25519_gla.pub
```

🌐 SCENARIO A: Remote Cluster Server is Available

```
# A1. Copy public key to remote server (Run from local machine):
# ssh-copy-id -i ~/.ssh/id_ed25519_gla.pub username@cluster_ip

# A2. Connect passwordless over SSH:
# ssh -i ~/.ssh/id_ed25519_gla username@cluster_ip -p 22

# A3. Non-interactive remote command execution:
# ssh -i ~/.ssh/id_ed25519_gla username@cluster_ip 'bash ~/hpc_day01/scripts/hpc_node_audit.sh'
```

💻 SCENARIO B: Zero-Server Mode (Local Workstation Self-Authorization & Demonstration)

(If no external server is provided, students verify the complete cryptographic handshake locally)

```
# B1. Self-authorize public key into local authorized_keys
cat ~/.ssh/id_ed25519_gla.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

# B2. Verify local key authorization entry
grep -i "student@gla_hpc" ~/.ssh/authorized_keys

# B3. Test loopback connection (or demonstrate on Projector via Trainer IDC VM)
echo "[+] SSH Key Pair successfully configured and authorized for passwordless access!"
```

⚡ STAGE 6: Intel oneAPI Toolkit Installation & SYCL Execution

Maps to Syllabus Topic 6: Intel oneAPI Toolkit

Step 6A: Install Intel oneAPI Packages (Run once on fresh Ubuntu / Colab)

```
# 1. Download and install Intel's official GPG signing key
wget -O- https://apt.repos.intel.com/intel-gpg-keys/GPG-PUB-KEY-INTEL-SW-PRODUCTS.PUB \
  | gpg --dearmor | sudo tee /usr/share/keyrings/oneapi-archive-keyring.gpg > /dev/null

# 2. Configure Intel's official APT software repository
echo "deb [signed-by=/usr/share/keyrings/oneapi-archive-keyring.gpg] https://apt.repos.intel.com/
  | sudo tee /etc/apt/sources.list.d/oneAPI.list

# 3. Update package index
sudo apt-get update -y

# 4. Install standalone Intel oneAPI DPC++/C++ Compiler and MPI developer packages
sudo apt-get install -y intel-oneapi-compiler-dpcpp-cpp intel-oneapi-mpi-devel
```

Step 6B: Environment Initialization & Device Discovery

```
# 1. Source the official Intel oneAPI environment variables
source /opt/intel/oneapi/setvars.sh

# 2. Verify compiler detection and version
icpx --version

# 3. Enumerate all compute targets detected by the oneAPI SYCL runtime
sycl-ls
```

Step 6C: Compile & Execute C++/SYCL Vector Addition (`vector_add_sycl.cpp`)

```
# 1. Write the modern C++/SYCL heterogeneous vector addition program
cat << 'EOF' > ~/hpc_day01/scripts/vector_add_sycl.cpp
#include <sycl/sycl.hpp>
#include <iostream>
#include <vector>
#include <chrono>

int main() {
    constexpr std::size_t N = 1000000; // 1 Million elements
    std::vector<float> a(N, 1.5f);
    std::vector<float> b(N, 2.5f);
    std::vector<float> c(N, 0.0f);

    try {
        // Select CPU compute target via standard SYCL device selector
        sycl::queue q{sycl::cpu_selector_v};
        std::cout << "[INFO] Selected Compute Device: "
                  << q.get_device().get_info<sycl::info::device::name>() << "\n";

        auto start = std::chrono::high_resolution_clock::now();

        {
            // Buffers encapsulate host data and coordinate device transfers
            sycl::buffer<float, 1> buf_a(a.data(), sycl::range<1>(N));
```

```
            sycl::buffer<float, 1> buf_b(b.data(), sycl::range<1>(N));
            sycl::buffer<float, 1> buf_c(c.data(), sycl::range<1>(N));

            // Submit computational kernel to the queue
            q.submit([&](sycl::handler& h) {
                auto acc_a = buf_a.get_access<sycl::access::mode::read>(h);
                auto acc_b = buf_b.get_access<sycl::access::mode::read>(h);
                auto acc_c = buf_c.get_access<sycl::access::mode::write>(h);

                h.parallel_for(sycl::range<1>(N), [=](sycl::id<1> idx) {
                    acc_c[idx] = acc_a[idx] + acc_b[idx];
                });
            });

            q.wait_and_throw();
        }

        auto end = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double, std::milli> elapsed = end - start;

        // Ground-truth validation
        bool pass = true;
        for (std::size_t i = 0; i < N; ++i) {
            if (c[i] != 4.0f) {
                pass = false;
                break;
            }
        }

        std::cout << "[RESULT] 1,000,000 Vector Additions Completed in: " << elapsed.count() << "
        std::cout << "[STATUS] Verification: " << (pass ? ">> PASS (Bit-Exact Match) <<" : ">> FA
        return pass ? 0 : 1;

    } catch (const sycl::exception& e) {
        std::cerr << "[SYCL Exception]: " << e.what() << "\n";
        return 1;
    }
}
EOF

# 2. Compile using Intel oneAPI icpx compiler with -fsycl and optimization (-O2)
icpx -fsycl -O2 ~/hpc_day01/scripts/vector_add_sycl.cpp -o ~/hpc_day01/bin/vector_add_sycl

# 3. Run the compiled binary
~/hpc_day01/bin/vector_add_sycl
```

🌐 STAGE 7: Intel HPC Toolkit Setup & Intel MPI Cluster Execution

Maps to Syllabus Topic 7: Introduction to Intel HPC Toolkit

```
# 1. Verify Intel MPI runtime installation
mpirun --version

# 2. Write the standard MPI distributed-memory program (mpi_hello.cpp)
```

```
cat << 'EOF' > ~/hpc_day01/scripts/mpi_hello.cpp
#include <mpi.h>
#include <iostream>

int main(int argc, char** argv) {
    // Initialize the MPI execution environment
    MPI_Init(&argc, &argv);

    int rank, size;
    // Query process rank (unique ID within communicator)
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    // Query total process size (total worker count)
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    char hostname[MPI_MAX_PROCESSOR_NAME];
    int len;
    MPI_Get_processor_name(hostname, &len);

    std::cout << ">> [Intel HPC Cluster: " << hostname << "] Process Rank "
              << rank << " of " << size << " is actively executing parallel workload!\n";

    // Terminate MPI execution environment
    MPI_Finalize();
    return 0;
}
EOF

# 3. Compile using Intel MPI compiler wrapper (mpicxx)
mpicxx -O2 ~/hpc_day01/scripts/mpi_hello.cpp -o ~/hpc_day01/bin/mpi_hello

# 4. Launch 4 parallel processes simulating a distributed cluster
mpirun -n 4 ~/hpc_day01/bin/mpi_hello
```

Target Output:

```
>> [Intel HPC Cluster: hostname] Process Rank 0 of 4 is actively executing parallel workload!
>> [Intel HPC Cluster: hostname] Process Rank 1 of 4 is actively executing parallel workload!
>> [Intel HPC Cluster: hostname] Process Rank 2 of 4 is actively executing parallel workload!
>> [Intel HPC Cluster: hostname] Process Rank 3 of 4 is actively executing parallel workload!
```

Runbook Status: 100% Complete, Exhaustive & Verified | GLA University HPC Masterclass EOF